

Static Code Analysis Using SonarQube and Jenkins

In the first of this two part series, we discussed the importance of static code analysis and the tools that can be used for it. In this article, we will discuss SonarQube integration with the Jenkins pipeline.

SonarQube checks code quality and code security to enable the writing of cleaner and safer code. It currently supports code analysis in 27 programming languages using different plugins available for the default standard rule set.

SonarQube is an automatic code analysis tool to find bugs, vulnerabilities and code smells in your source code. It can be integrated with the existing development workflow to enable continuous code analysis across project branches and pull requests. It can also be integrated directly into IDEs (Eclipse, VS Code, Visual Studio and IntelliJ) to find code related issues while developing code.

The workflow which should be followed for continuous static code analysis is given in Figure 1.

In the figure, the standard development process is shown through the following steps:

1. The developer should use SonarLint to receive immediate feedback in the IDE while coding, and then commit the code to the source code repository (GitHub, GitLab, Azure DevOps, Bitbucket).
2. Continuous integration (CI) pipeline should be triggered to produce builds, run unit tests and to analyse the source code with the help of the SonarQube scanner. CI tools that can be easily integrated with SonarQube analysis are Jenkins, GitLab, Azure DevOps, Bitbucket, and others.
3. Once the pipeline executes the analysis, the scanner publishes the results to the SonarQube server. Then the latter provides feedback to developers via the SonarQube interface, email, in-IDE notifications (via SonarLint), and decoration on pull requests. This article focuses only on integrating SonarQube analysis with the Jenkins pipeline.

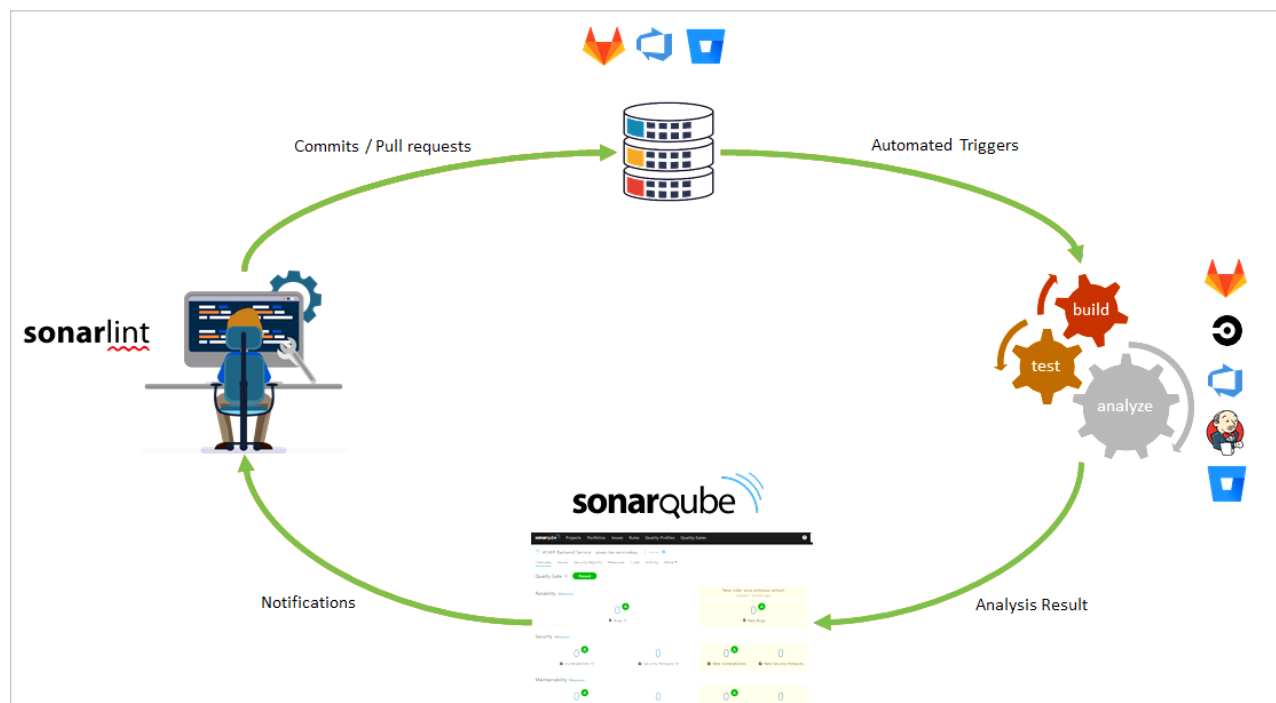


Figure 1: Standard development process with SonarQube